

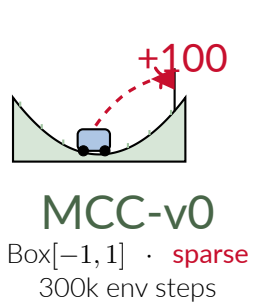
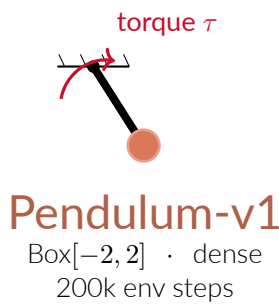
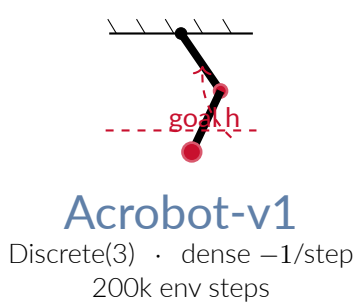
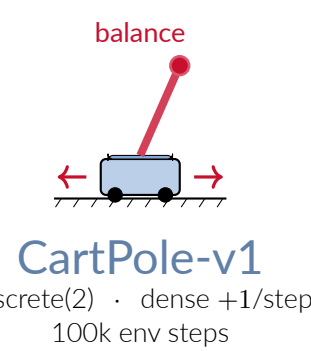
A Comparative Study of Deep RL Algorithms: DQN, PPO, SAC, TD3

Kevin Abou Jaoude Youssef Dib Fuad Khuri

EE-568 Reinforcement Learning · EPFL · Spring 2026

EPFL

1. Four classic-control environments



2. Setup and taxonomy

We re-implement four canonical deep RL algorithms from scratch in PyTorch (no `stable-baselines3`) and benchmark them with 3 seeds each, 95% bootstrap CIs over seeds, and four pre-registered ablations.

	on-policy	off-policy
value-based	(none)	DQN
policy-based	PPO	SAC, TD3

Networks. (64, 64) MLPs for DQN, PPO; (256, 256) for SAC, TD3 (canonical paper widths). **Hardware.** RTX 3070 + EPFL gnoto V100.

3. Algorithms

DQN [1]

value-based, off-policy, discrete:

$$\mathcal{L} = \mathbb{E}[\ell_{\text{Hub}}(Q_\theta - r - \gamma(1-d) \max_{\theta'} Q_\theta)]$$

ϵ -greedy, target sync every τ steps, Huber loss.

PPO [2]

policy-based, on-policy:

$$\mathcal{L} = -\mathbb{E}[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 \pm \epsilon) \hat{A}_t)]$$

GAE advantages, clipped surrogate as a soft trust region.

SAC [3]

off-policy, max-entropy:

$$J = \mathbb{E}[\sum_t \gamma^t (r_t + \alpha \mathcal{H}[\pi(\cdot|s_t)])]$$

Squashed-Gaussian actor, auto α via $\log \alpha$, twin Q.

TD3 [4]

off-policy, deterministic:

$$y = r + \gamma(1-d) \min_i Q_{\phi_i}(s', \pi_{\theta} + \tilde{\epsilon})$$

Twin critics, target smoothing $\tilde{\sigma}=0.2$, delayed actor.

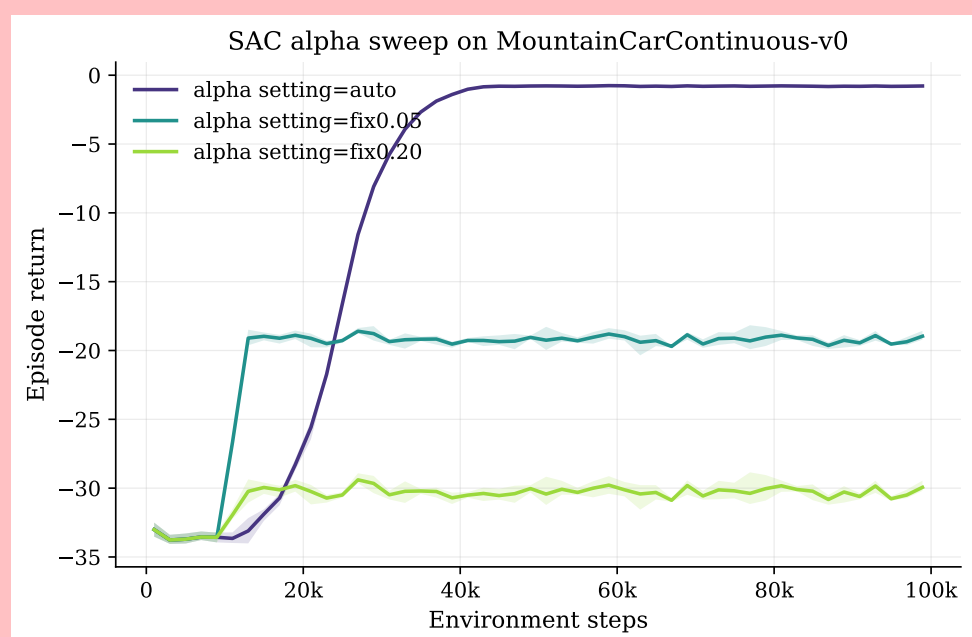
4. Implementation details that mattered

- DQN:** target-sync sweep on CartPole gives a **U-shape** ($\tau \in \{100, 1000, 5000\}$). The canonical $\tau=1000$ sits at the edge of stability, which explains the seed-bimodality.
- PPO:** at Pendulum truncation we fold $\gamma V(s_{t+1})$ into the reward so GAE accounts for the value of the truncated state; raw **done** flags bias returns by ~ 30 .
- SAC:** the $\tanh$ -Jacobian log-prob correction $-\sum_i \log(1 - \tanh^2 u_i)$ is mandatory; without it the entropy term is silently wrong and training never moves off the warmup floor.
- TD3:** the smoothed target action needs two clips, on the noise (± 0.5) and on the action range. Skipping the outer clip sticks the actor on the boundary.
- All four:** bootstrap mask uses **terminated** only, not **terminated or truncated**.

5. Pre-registered ablations: turning the canonical knobs

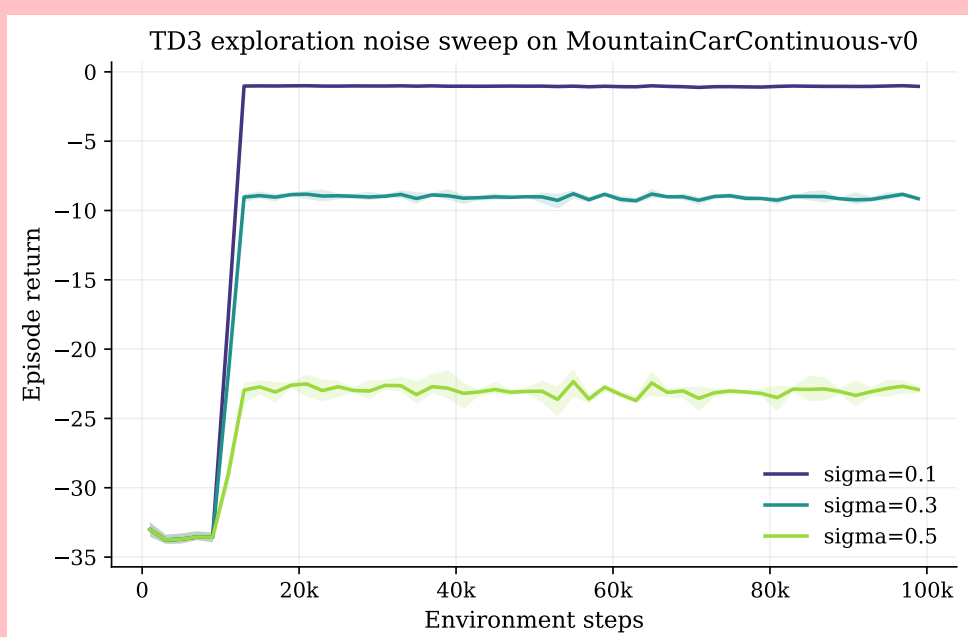
Four pre-registered sweeps, one per algorithm, 3 seeds each.

Headline (MCC, sparse reward): turning the off-policy exploration knobs (SAC α , TD3 σ) does not rescue them.



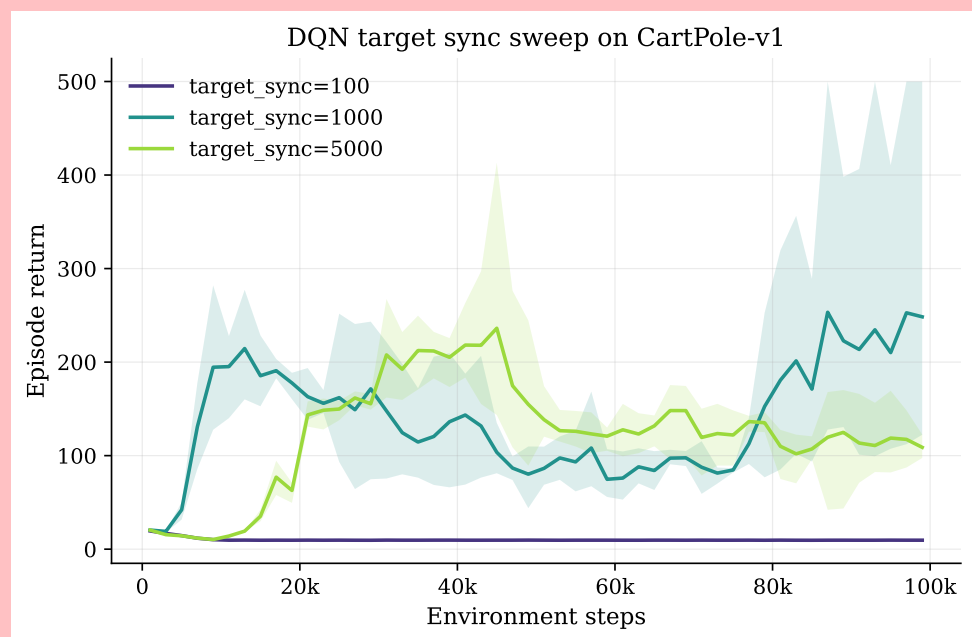
SAC α sweep on MCC.

Auto, 0.05, 0.20: three failure modes, 0 goals / 9 seeds.



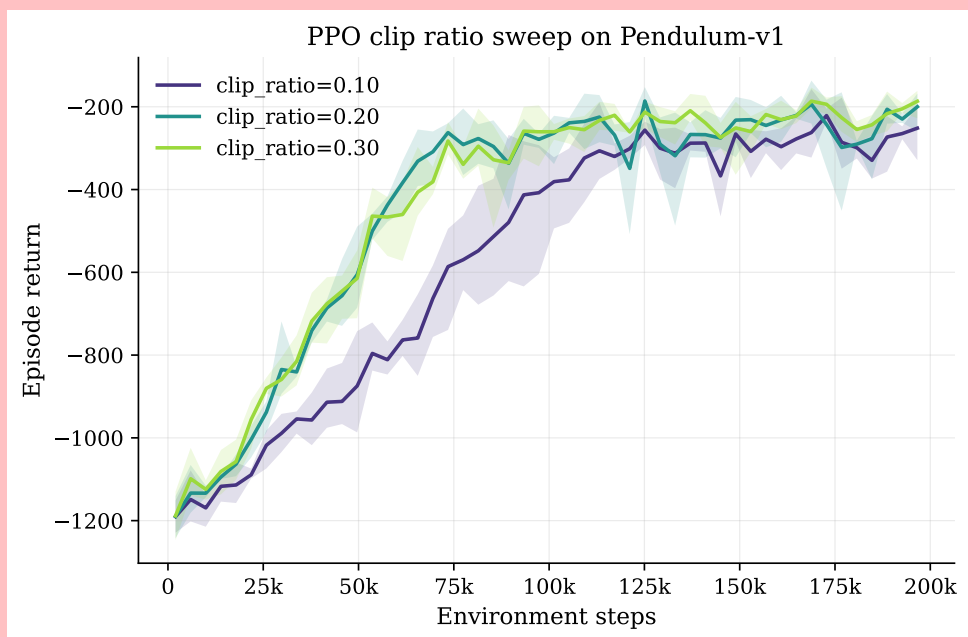
TD3 σ sweep on MCC.

$\sigma \in \{0.1, 0.3, 0.5\}$: more noise is worse.



DQN target-sync, CartPole.

U-shape; $\tau=1000$ at the edge of stability.



PPO clip ratio, Pendulum.

$\epsilon=0.1$ too conservative; 0.2 & 0.3 tied.

Why off-policy continuous methods fail on MCC. Entropy regularization (SAC) and additive Gaussian noise (TD3) drive uniform coverage of the *local* action set, not directed exploration in state space. PPO's on-policy rollouts occasionally produce a long correlated action sequence that crosses the hill; off-policy replay samples are decorrelated and cannot.

6. Limitations & open questions

- Seed count.** 3 seeds per (algo, env). More would tighten CIs but cost is super-linear in our compute budget; the 3-seed bootstrap CIs are wide enough to keep our claims honest.
- Network widths fixed.** (64, 64) for DQN, PPO and (256, 256) for SAC, TD3 (canonical paper defaults). The actor-parameter gap is a hyperparameter choice, not an algorithmic property, and we say so when comparing model sizes.
- Sparse reward = MCC only.** Generalizing the headline (off-policy exploration gap on sparse reward) needs a second sparse-reward continuous benchmark, for example **Reacher** with a binary success reward or a sparsified **Pendulum**.
- No exploration-baseline comparison.** Natural next step: RND or NoisyNet on top of SAC/TD3 and re-run the MCC ablation. Does directed exploration close the gap that local entropy/noise cannot?

Key numbers at a glance

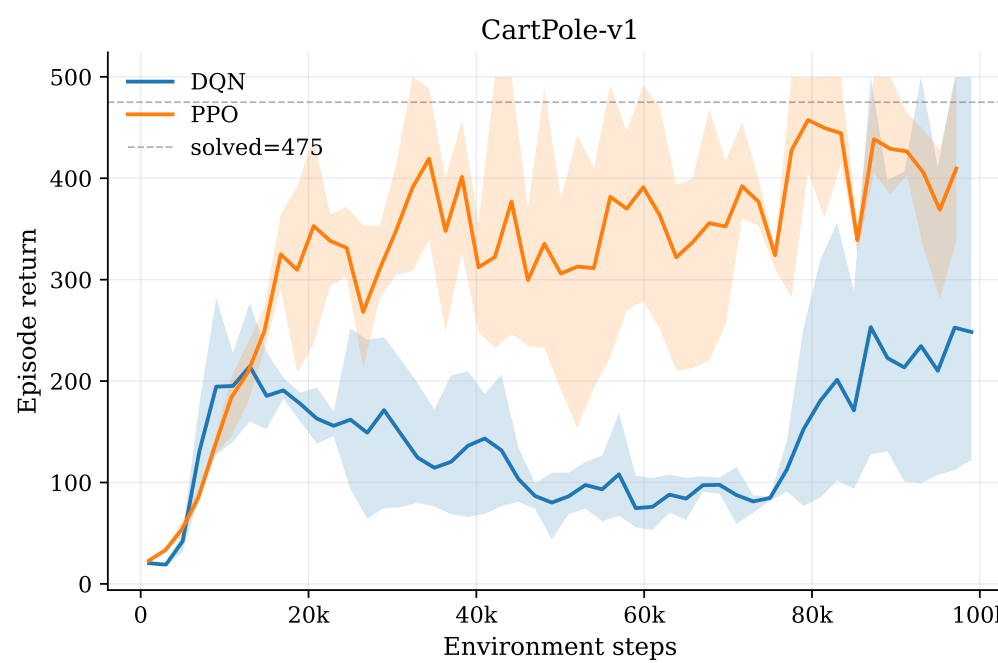
13×

SAC sample-efficiency over PPO on Pendulum

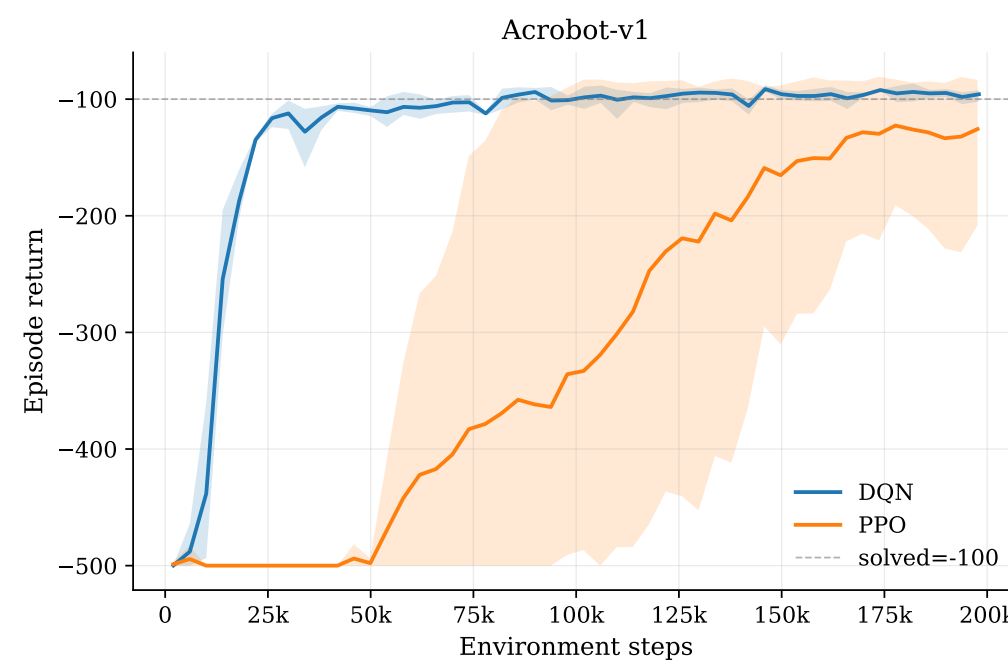
0/9

Goal discoveries by SAC across α -sweep seeds on MCC

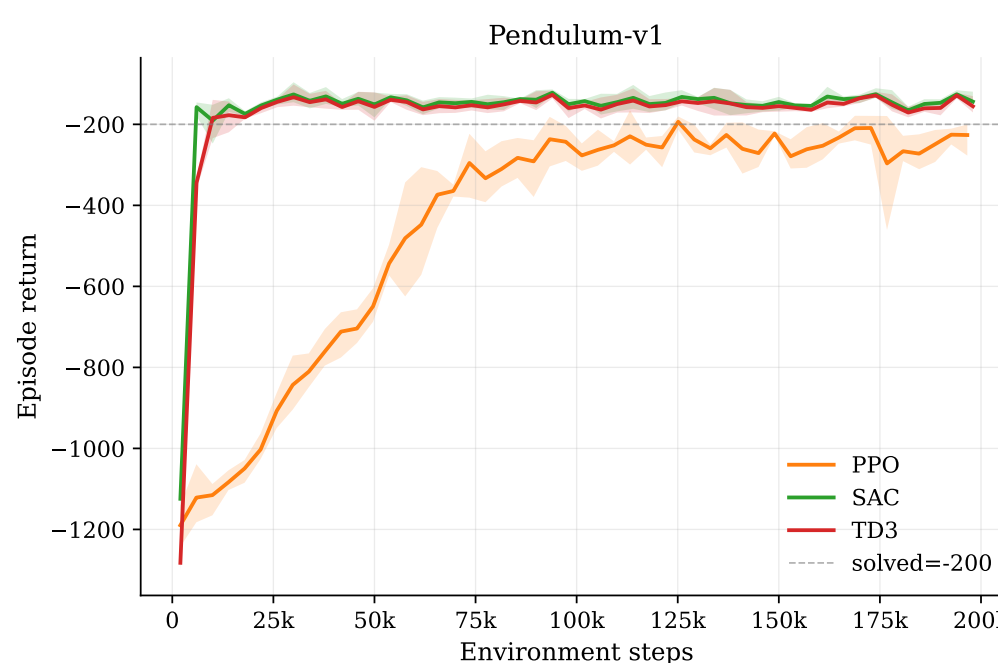
A. Learning curves: one figure per environment



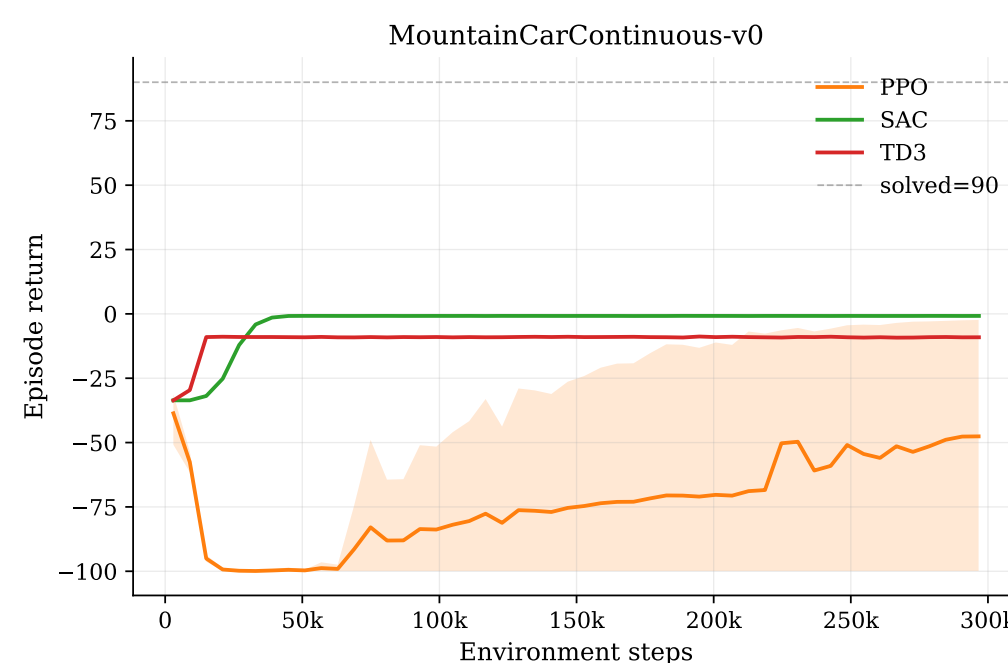
CartPole. DQN bimodal (1/3 solve); PPO reliable, 380, CI [350, 402].



Acrobot. DQN solves cleanly in ~ 30 k steps; one PPO seed collapses.

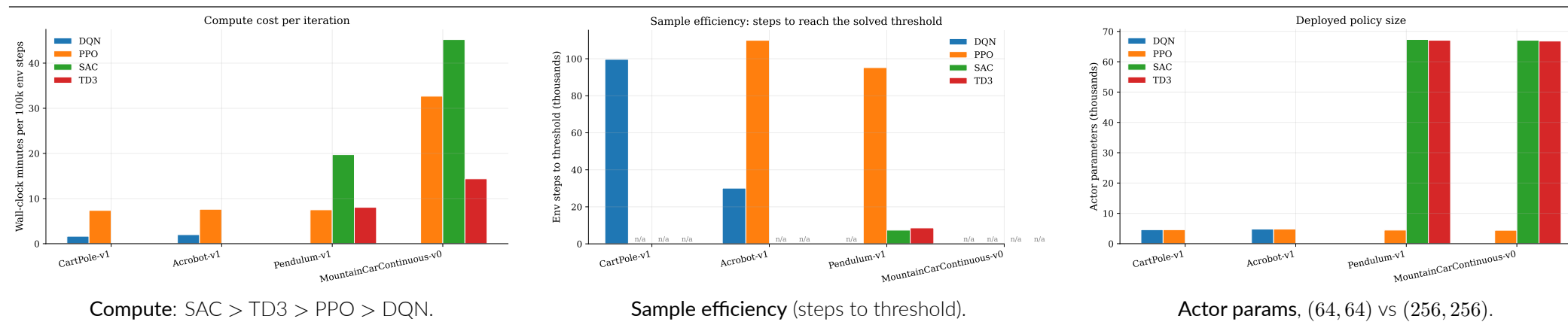


Pendulum. SAC and TD3: $\sim 13\times$ more sample-efficient than PPO.



MCC, sparse. Only PPO ever finds the goal (1/3 seeds, -2.5).

B. Cost, sample efficiency, model size

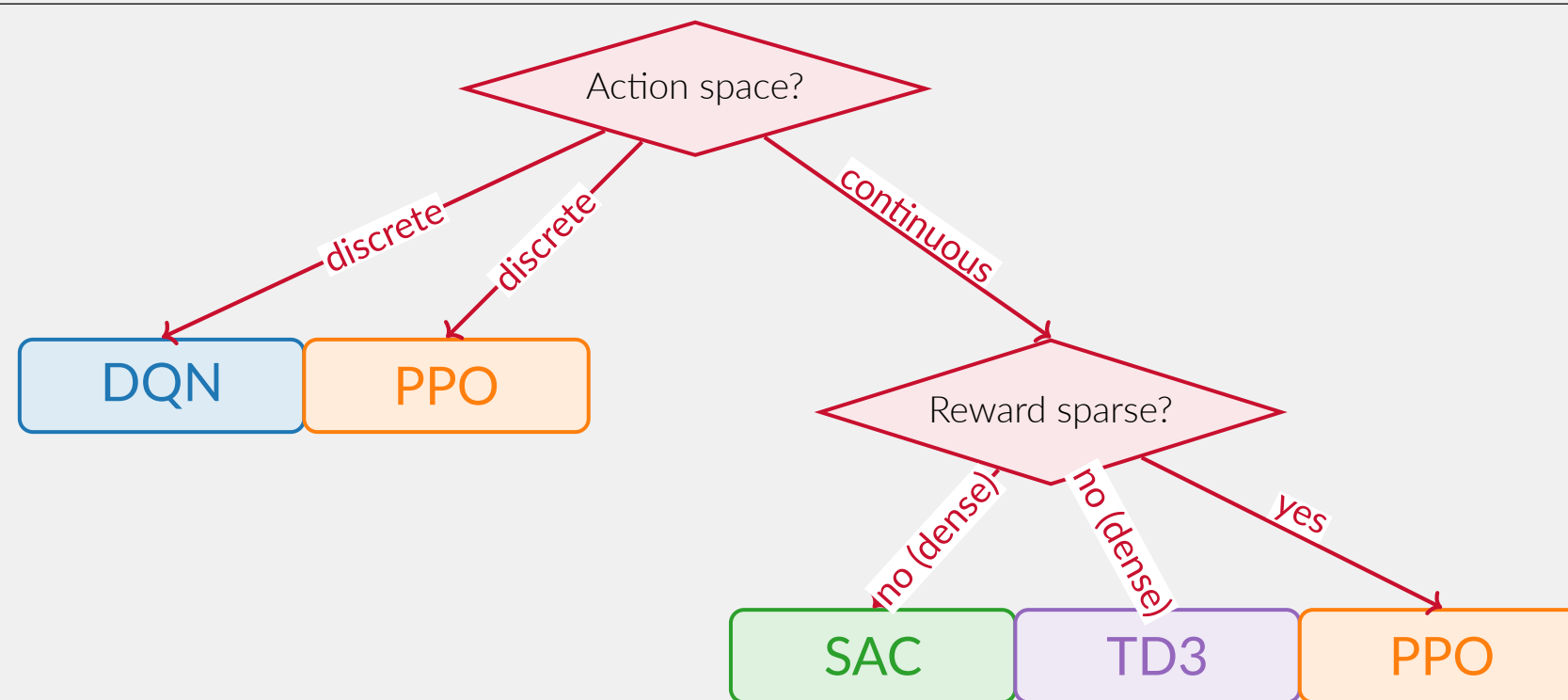


Compute: SAC > TD3 > PPO > DQN.

Sample efficiency (steps to threshold).

Actor params. (64, 64) vs (256, 256).

C. When to use which algorithm



- Discrete actions:** DQN wins on Acrobot, bimodal on CartPole; PPO is more reliable.
- Dense-reward continuous:** SAC and TD3 dominate, 12.9 $\times$ over PPO on Pendulum.
- Sparse-reward continuous:** PPO is the only one to find the MCC goal; the failure is structural, not a tuning issue.

Pick SAC or TD3 first on continuous control; switch to PPO (or add explicit exploration bonuses) the moment reward becomes sparse.

References & project artefacts

- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv:1312.5602*, 2013.
 - John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.
 - Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep rl with a stochastic actor. In *ICML*, 2018.
 - Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *ICML*, 2018.
 - Mark Towers, Jordan K. Terry, Ariel Kwiatkowski, et al. Gymnasium, 2023.
- Artefacts:** PyTorch from scratch (no `stable-baselines3`); per-(algo, env) YAML in `configs/`; curves, seed grids, ablations and bar charts under `plots/`; bootstrap-CI scoreboard in `plots/scoreboard.{csv, tex}`.